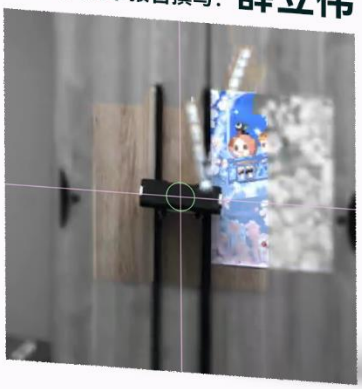
英雄图传技术开源

RMUC 2026 部署模式低带宽落点观察

技术研发/报告撰写：薛立伟



前言

在 2026 赛季中，英雄仅在远程优先，并进入部署模式下才可解锁 16.5m/s 的高弹速。此时英雄将会失去图传画面。此时，唯一可用的机器人上行数据链路为机器人发送给自定义客户端的数据。本开源方案将会以此入手，提出在极有限带宽下实时传输视频画面的方案。

本赛季中，五大湖联合大学虽然顶着巨大的不利条件做出了全阵容，但依旧遗憾无缘 RMUC 的赛场。我们希望借此开源的机会，让研发成果惠及更多有机会上场的队伍。

如果本文对您有帮助，或者您参考本文开发了 RMUC 比赛用客户端，请私信笔者告知您的学校名称。**这对笔者有很大意义。**如果您对过洞吊射英雄的机械设计方案感兴趣，也欢迎移步五大湖联合大学的相关机械开源。

本开源报告采用 CC-BY-SA 4.0 协议。这意味着本开源文档可以在标注来源的前提下，以**任何用途**被自由共享、演绎、使用。由本报告修改而来的文档需要以相同协议开源。

目录

Part 0. **图传需求分析** 3
带宽和图像效果 3
测试基准 3
Part 1. **视频编码方案实现** 4
相机画面采集 4
ROI/降采样 5
静态信息剔除 6
多帧弹道拖影 7
编码 8
解码和显示 9
Part 2. **实机测试** 10

注：本报告中所引用的第三方图片和文档不在开源许可范围内。

Part 0. 图传需求分析

带宽和图像效果

在部署模式下，唯一可用的上行数据链路为裁判系统 0x0310 号消息。此条消息通过图传链路传输至自定义客户端。

在初始版本的通信协议文档中，此链路单个数据包 payload 为 150Byte：

0x0310	150	机器人发送给自定义客户端的数据，频率上限为 50Hz	己方机器人→图传链路→对应操作手选手端连接的自定义客户端	图传链路

在 V1.3 版本通信协议文档中，此链路单个数据包 payload 大小进一步提升至 300Byte：

0x0310	300	机器人发送给自定义客户端的数据，频率上限为 50Hz	己方机器人→图传链路→对应操作手选手端连接的自定义客户端	图传链路

注：本文中的方案最初基于初始版本的协议研发，在新版双倍带宽协议下效果更好。

消息的发送频率上限为 50Hz。因此，链路最大比特率为：

$$300\text{Byte} \times 50\text{Hz} = 15\text{kB/s}$$

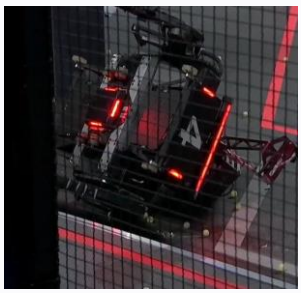
考虑到裁判系统过高负载下的丢包问题以及安全冗余，实际视频流带宽应控制在 12kB/s 左右。此外，由于长焦相机画面中弹丸下落速度极快，系统还需要实现高帧率传输，并在必要时对弹丸路径进行高亮。红蓝灯条和绿色的发光大弹丸在彩色画面中与黑色背景形成鲜明对比，图传中支持彩色对操作手也有所帮助。

测试基准

在正式部署到机器人上进行测试前，需要使用样例视频用于测试编码方案。从信息论视角来看，视频压缩算法，或者说任何有损/无损压缩算法的压缩率，都高度取决于压缩前内容的信息量。在赛季初，笔者观察到数支队伍都采用了非吊射视角的视频录像来测试算法，并得出图传无法实现的错误结论。对于吊射图传这样高度专业化且信息量相对低的应用场景，使用错误的测试视频只会得出无意义的测试数据。

下面是一些合适和不合适的测试画面样例：

合适

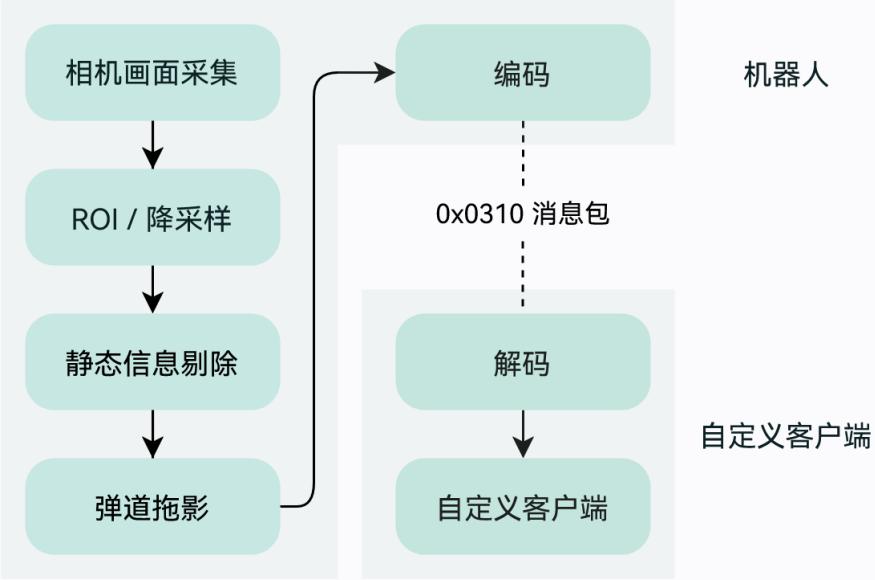


不合适

非常不合适

Part 1. 视频编码方案实现

图传系统的架构如下。



相机画面采集

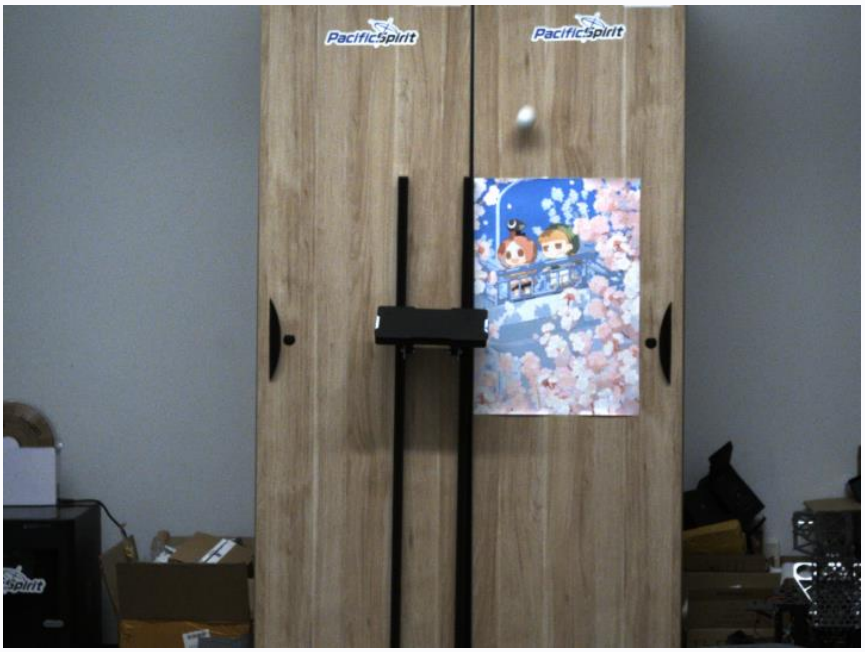
出于兼容自瞄系统的考虑，我们的系统使用了海康 MV-CS016-10UC 工业相机拍摄靶标附近的画面。理论上任何满足分辨率和帧率的相机均可使用；这里使用海康相机，是为了拍摄的画面可以同时用于吊射自瞄。对于 MV-CS016 系列相机，我们建议搭配不小于 35mm 的长焦镜头。

考虑到弹道下坠，相机安装时需要有较大的俯角；经验值可以取俯角 40 度，并根据实测微调。设计相机安装位时，最好考虑到未来可能的俯角修改。

与自瞄识别装甲板不同，吊射图传需要保证人眼的识别度。建议使用 60fps 帧率配合 10ms 以上曝光时间，较大的光圈，并开启相机自带的 Gamma 矫正。相机帧率并非越大越好！除去高帧率对 MiniPC 的额外性能开销外，双相机架构可能争抢有限的总线带宽，造成

需求高帧率近战自瞄相机掉帧。如果机器人上的相机数量大于 1，尽量将相机连接到不同的 USB 控制器上（单个 USB Root Hub 可能对应多个 USB 口，这些 USB 口之间共享带宽）。

下面为我们搭建的模拟基地大装甲板测试场景。背景由较为纯净的白墙和较为复杂插画海报组成。图为相机采集到的未经处理的画面。 画面中有一发大弹丸已命中装甲板，正在向上弹起。

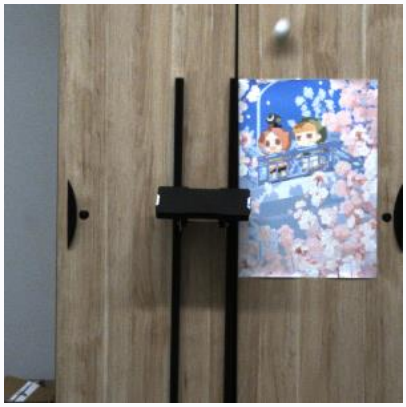


相机采集到的原始画面

ROI/降采样

在采集到画面后，首先要对画面取 ROI 并降采样到较低分辨率。在旧版串口协议下，传输的分辨率建议不高于 300x300。在新版协议下，建议不高于 400x400。

这里以 300x300 的分辨率为例，右图为取 ROI 并降采样后的画面。



取 ROI 并下采样后的画面

静态信息剔除

上文中已经提到，任何有损或者无损压缩算法的压缩效果，都取决于视频本身含有的信息量。对于单帧数据而言，画面越复杂，信息量越大；对于多帧数据来说，帧之间画面的变化越大，变化方式越没有规律，信息量越大。视频编码本质是一种有损压缩操作；编码器会尝试做出取舍，丢弃其认为的相对不重要的信息来减小数据量。

问题在于，通用的视频编码器并不知道在 RM 场景下，哪些信息是重要的，哪里不重要。为了提高编码效率，我们要先手动剔除一部分冗余画面信息，以得到更高的编码效率。

图传目标是帮助操作手校准弹丸落点。因此...

重要的信息是：装甲板位置，弹丸轨迹。

不重要的信息是：装甲板附近静止不动的背景。

为了实现这一目标，我们可以针对性的剔除静态背景的**高频信号**和**色彩信号**，仅保留低频的亮度信号。高频信号由高斯模糊（本质是低通滤波器）去除，色彩信号由取灰度操作去除。

首先，将当前帧和历史帧取灰度。接下来，将两帧相减，得到两帧之间每个像素的亮度变化量，并阈值化，得到二值运动掩码。运动掩码接下来进行一次较小的腐蚀去除噪点，再进行一次较大的膨胀扩展边界。最后，根据运动掩码，对静态部分应用高斯模糊+灰度以去除高频和颜色信息。为了保证画面中间的装甲板不被影响，画面中心的方形区域不做处理。

右图为处理后的效果。

剔除静态信息后的画面

为了进一步说明信息剔除对压缩效果的影响，我们可以进行一个试验。以下三张图片中，最左边的是未压缩过的原图；中间和右边的图经过了相同的 JPG 算法压缩到相同的 20kB 大小。中间的图片直接压缩，右边的图片通过信息剔除，强调中间区域的画质。

让我们重点为关注中间部分：

直接压缩

剔除外围信息

可以看到非常明显的画质区别。相比右侧，左侧未处理直接压缩的图中，色度信息几乎全部丢失了，只剩下大块的黄色和粉色色块。亮度信息保留的稍好，但人物的线条也非常扭曲模糊。

H264 算法的帧内压缩和 JPG 原理高度类似，都基于 DCT。实践中，对于超低码率下的 H264 算法来说，信息剔除带来的关键区域画质提升大得多，直接决定了画面是否可用。

多帧弹道拖影

利用图传校射的另一个难点为，高速下落的大弹丸在画面中出现的时间非常短暂。考虑到工业相机的高帧率和短曝光时间，画面中弹丸的运动模糊比较小。为了让操作手能够看清弹丸轨迹，图传系统需要对弹丸显示拖影。

拖影的原理较为简单：注意到白色的大弹丸相对背景环境亮度较高（比赛中的发光大弹丸来说就更亮了），我们直接将历史帧中的运动部分逐像素进行 max 运算即可。由于发射或者云台移动也会导致画面运动，需要设置拖影阈值。如果画面中的运动像素大于某个百分比，暂时禁用拖影。

拖影后的画面效果如下。可以清楚看到弹丸命中后弹起的轨迹。

加入弹道拖影之后的画面

编码

在本赛季规则发布后，笔者看到了诸多队伍尝试研发高效的图传压缩算法，包括简单的帧间压缩，色度采样甚至矢量化；也有少数队伍开始探索有效的帧间压缩算法。从个人学习和成长的角度来说，这些努力值得肯定；但是从工程和比赛的角度来说，重复造轮子的行为是**完全不必要**的。从早期的 MPEG 压缩算法，到成熟的 H264，再到 HEVC/AV 等现代压缩算法，视频编码标准已经经历了长久的研究和工程优化。况且，这些编码格式已有广泛的硬件支持和大量高性能的编解码器实现。

出于实现简单和支持良好的考虑，笔者在赛季出开发时使用了 H264 作为编码方案。H265 和 AV1 等更新的格式在低码率下性能更佳，代价是更高的处理器性能开销。

笔者使用 GStreamer 进行编码（用 FFmpeg 的库的时候频繁出现内存错误，实在没能力优化了），参数如下表。在一台 U9-285H 处理器的笔记本电脑上，纯 CPU 编码时核心占用为 150%（Linux top 命令查看），延迟在 700ms 左右。在新版本裁判系统协议将可用带宽翻倍后，读者如果感兴趣，可以尝试舍弃部分编码效率，使用延迟更低的编码配置。

帧率	60 FPS
GOP 关键帧间隔	>200
B 帧	4

sync-lookahead 前瞻帧（影响延迟）	20
CFR 固定帧率	强制
SPS/PPS	每个关键帧
Preset 预设	9 (Veryslow)
目标码率	10kB/s
AQ 自适应量化	Mode=2, strength=1.2
Trellis 率失真优化	2

这里的参数有不少优化空间，欢迎读者尝试并提供更多反馈。

解码和显示

使用 GStreamer 解码并附加上去辅助瞄准用的准心。下图是解码后的画面。

解码之后的画面

Part 2. 实机测试

您可以在 RoboMaster 社区开源贴或者笔者的哔哩哔哩主页 (UID: 310236726) 找到样例代码与演示视频。